Teste Técnico Júnior — Automação de Migração de Planos

NetVila Telecom · Processo seletivo de desenvolvimento

Visão geral

O candidato deve construir, com entrega até **23h59 (horário de Brasília) de domingo, 18/10/2026**, uma automação em JavaScript ou TypeScript que lê um arquivo CSV de migrações de plano e atualiza os contratos de internet no banco de dados.

A empresa fictícia **NetVila Telecom** recebe periodicamente uma planilha da área comercial pedindo a troca de plano de alguns clientes. Hoje isso é feito manualmente. A automação deve aplicar as trocas válidas e gerar um relatório com cada contrato que não pôde ser atualizado e o motivo.

Você receberá junto com este enunciado dois arquivos:

- `seed.sql` — cria as tabelas `planos` e `contratos` e insere os dados de exemplo.
- `migracoes_planos.csv` — o arquivo de entrada que a automação deve processar.

Modelo de dados

O banco é SQLite e tem duas tabelas: cada contrato aponta para um plano por `contratos.plano_id → planos.id` (relacionamento N:1).

Tabela `planos`

Coluna	Tipo	Descrição
id	INTEGER, PK	Identificador do plano
nome	TEXT	Nome comercial do plano
status	TEXT	`ativo` ou `inativo`

Tabela `contratos`

Coluna	Tipo	Descrição
id	INTEGER, PK	Número do contrato
nome	TEXT	Nome do cliente
telefone	TEXT	Telefone de contato
plano_id	INTEGER, FK → planos.id	Plano atual do contrato
status	TEXT	`ativo`, `cancelado` ou `nao_instalado`
data_instalacao	TEXT (YYYY-MM-DD), pode ser nulo	Nulo quando o contrato ainda não foi instalado

Planos cadastrados

id	nome	status
1	Fibra 100 Mega	ativo
2	Fibra 300 Mega	ativo
3	Fibra 500 Mega	ativo
4	Fibra 50 Mega (legado)	inativo

A tabela `contratos` vem com 15 registros: 9 ativos, 3 cancelados e 3 não instalados. Os dados completos estão no `seed.sql`.

Arquivo CSV de entrada

O CSV usa vírgula como separador, tem cabeçalho e três colunas: `id` (id do contrato), `de_plano` (id do plano atual) e `para_plano` (id do novo plano).

```
id,de_plano,para_plano
1,1,2
3,1,3
4,3,1
6,4,2
7,2,4
9,2,3
10,1,1
99,1,2
11,2,7
12,3,2
5,2,
14,1,3
14,3,2
8,1,2
15,abc,3
```

A automação deve funcionar com qualquer arquivo no mesmo formato, não só com este exemplo. O caminho do arquivo deve ser recebido como argumento na linha de comando.

Regras de negócio

Só contratos com status `ativo` ou `nao_instalado` podem trocar de plano. Cada linha do CSV passa pelas validações abaixo, nesta ordem; a primeira que falhar define o motivo e a linha não é aplicada.

Ordem	Validação	Motivo no relatório
1	`id`, `de_plano` e `para_plano` preenchidos e numéricos	Linha inválida
2	O `id` do contrato não apareceu antes no mesmo arquivo	Contrato duplicado no arquivo
3	O contrato existe na tabela `contratos`	Contrato não encontrado
4	O contrato está `ativo` ou `nao_instalado`	Contrato cancelado
5	`de_plano` é igual ao `plano_id` atual do contrato	Plano de origem não confere
6	`para_plano` existe na tabela `planos`	Plano de destino não encontrado
7	`para_plano` está com status `ativo`	Plano de destino inativo
8	`para_plano` é diferente de `de_plano`	Contrato já está no plano de destino

Regras complementares:

- Na regra 2, a primeira ocorrência do `id` segue o fluxo normal; as seguintes são rejeitadas.
- Um contrato pode sair de um plano inativo para um plano ativo. Apenas o destino precisa estar ativo.
- A atualização altera somente `plano_id`. Nenhuma outra coluna do contrato muda.
- Uma linha rejeitada não pode interromper o processamento das demais.

Saída esperada

Ao final da execução, os contratos válidos estão com o novo `plano_id` no banco e existe um relatório CSV com todas as linhas rejeitadas.

O relatório deve se chamar `relatorio_nao_atualizados.csv` e ter estas colunas:

```
linha,id,nome,de_plano,para_plano,motivo
```

- `linha` é o número da linha no CSV de entrada (a primeira linha de dados é a 2).
- `nome` vem do banco; fica vazio quando o contrato não existe.
- `de_plano` e `para_plano` repetem os valores recebidos, mesmo que inválidos.

No terminal, a automação deve exibir um resumo simples: total de linhas lidas, total atualizado e total rejeitado.

Requisitos técnicos e entrega

O prazo de entrega é 23h59 (horário de Brasília) de domingo, 18/10/2026, e a entrega é feita pelo link de um repositório público no GitHub, criado por você para este teste e enviado ao recrutador. O teste foi dimensionado para cerca de 3 dias de trabalho; o prazo maior dá folga para quem trabalha ou estuda durante a semana.

Obrigatório:

- Node.js 18 ou superior, com JavaScript ou TypeScript.
- Banco SQLite criado a partir do `seed.sql` (sugestão: `better-sqlite3` ou `sqlite3`).
- Execução por linha de comando, por exemplo `npm start -- ./migracoes_planos.csv`.
- Um script ou comando para recriar o banco do zero com o `seed.sql`.
- `README.md` descrevendo como executar o projeto: instalação das dependências, criação do banco e execução da automação, além das decisões tomadas. Esse passo a passo faz parte da avaliação.

Livre escolha: bibliotecas de leitura de CSV, estrutura de pastas e uso de ORM. Não é necessário interface gráfica nem API HTTP.

Sugestão de ritmo para o candidato:

1. Dia 1: preparar o projeto, criar o banco e ler o CSV.
2. Dia 2: implementar as validações e a atualização.
3. Dia 3: gerar o relatório, revisar o código e escrever o README.

Critérios de avaliação

A avaliação é qualitativa: não há nota, e cada item abaixo descreve o que será observado na entrega.

Critério	O que observamos
Regras de negócio	As 8 validações aplicadas na ordem certa e somente os contratos válidos atualizados
Relatório	Arquivo gerado com todas as linhas rejeitadas e o motivo correto de cada uma
Organização do código	Funções pequenas, nomes claros e separação entre leitura do CSV, validação e escrita no banco
Instruções de execução (README)	O README descreve, passo a passo, como instalar, criar o banco e executar o projeto. O avaliador segue exatamente essas instruções, e o projeto deve rodar na primeira tentativa
Uso do Git	Commits pequenos e com mensagens descritivas, mostrando a evolução do trabalho

Diferenciais: qualquer adição que vá além do obrigatório conta a favor. Alguns exemplos:

- Geração de log da execução.

- Testes automatizados.
- Qualquer outra melhoria que você julgar útil. Explique no README o que fez e por quê.

Uso de inteligência artificial

Não é permitido usar ferramentas de IA (como ChatGPT, Claude, Copilot ou similares) para gerar código neste teste. Consultas a documentação, fóruns e buscadores são livres. Na entrevista de devolutiva, você poderá ser convidado a explicar e alterar o código ao vivo.

Dúvidas

Dúvidas sobre o enunciado podem ser enviadas ao recrutador durante o prazo. Quando a regra não cobrir algum caso, escolha um comportamento e registre a decisão no README.